1 Evaluation

1.1 Experiment 1: CMO-RL versus CHEHAB RL with Rotation Key Selection

This experiment asks whether a single CMO-RL policy can satisfy a noise budget that the original compiler violates, without giving up the latency that the original compiler achieves. We compare CMO-RL with the original CHEHAB RL optimizer [**?**] followed by its rotation key selection pass. We fix one noise budget and one preference vector, compile every benchmark with both systems, and measure three quantities for each compiled circuit: the noise it consumes, its execution time, and the size of the rotation keys it requires.

Baseline. CHEHAB RL rewrites a program by applying term rewriting rules selected by a PPO agent. Its reward is derived from a single cost function that combines operation cost, circuit depth, and multiplicative depth. This cost function targets execution time only. CHEHAB RL is therefore unaware of two properties that matter in deployment: the number of distinct rotation keys that the circuit requires and the noise that the circuit accumulates. To control the first property, CHEHAB RL applies a post-optimization pass that we call rotation key selection (RKS). RKS counts how often each rotation offset is used and rewrites infrequent rotations as chains of power-of-two rotations using their non-adjacent form (NAF). For example, a rotation by 12 becomes a rotation by 8 followed by a rotation by 4, which removes the dedicated key for offset 12 when the keys for offsets 8 and 4 are already present. RKS stops once the number of keys falls below a fixed threshold. RKS reduces key memory, but it runs after the agent has fixed the structure of the circuit, and it does not consider noise. We refer to the complete baseline as CHEHAB RL+RKS. [TODO: State the RKS key threshold used here and how it was chosen. For a fair comparison we suggest either the default threshold of the original implementation or a threshold equal to the key count produced by CMO-RL.]

CMO-RL. CMO-RL is a single policy conditioned on two inputs: a preference vector $\vec{w} = (w_{exec}, w_{keys})$, which sets the relative weight of execution time and rotation key memory, and a noise budget $\mathcal{B}$, which bounds the noise that the compiled circuit may consume (section 1). The policy chooses rewrites for both objectives jointly. It can therefore select a vectorization strategy that uses fewer rotation offsets, instead of decomposing rotations after the strategy has been fixed. We evaluate the raw output of the policy, with the safety rollback described in section 1 disabled. As a result, every constraint satisfaction we report comes from the policy itself and not from the rollback mechanism. [TODO: Confirm that rollback is disabled for Exp. 1; if it is enabled, say so and report how often it was triggered.]

Setup. We set the noise budget to $\mathcal{B} = 230$ bits of consumed BFV noise. We chose this value because it lies in the range where the circuits produced by CHEHAB RL+RKS exceed the budget for a majority of the benchmarks, while satisfying it is still possible: it is the tightest of the budgets that CMO-RL was trained on. [TODO: Confirm the budget value and its meaning (consumed noise bits as estimated by the noise model vs. the remaining noise budget measured by SEAL for fixed encryption parameters $(N, \log q)$). State the encryption parameters.] We set the preference vector to $\vec{w} = (w_{exec}, w_{keys}) = (0.9, 0.1)$. This preference keeps execution time as the primary objective while assigning a non-zero weight to rotation key memory; it is the preference closest to the single objective of CHEHAB RL that still asks CMO-RL to co-optimize memory. CHEHAB RL+RKS has no preference input and is run unchanged. We evaluate both systems on the 14 benchmarks listed in table 1, none of which appears in the training set. [TODO: Final benchmark list; the list below is a placeholder subset of the thesis benchmarks.] For every compiled circuit we report: (1) the consumed noise and whether it exceeds $\mathcal{B}$ (a *violation*); (2) the execution time, measured with [TODO: backend, e.g., SEAL BFV] on [TODO: hardware], as the [TODO: median] of [TODO: n] runs; and (3) the number of distinct rotation keys and their serialized size in MB. A serialized Galois key occupies approximately 18 MB in our setting [TODO: confirm for the parameters used here].

Constraint satisfaction. fig. 1(a) and table 1 show that CHEHAB RL+RKS exceeds the noise budget on 10 of the 14 benchmarks, whereas CMO-RL satisfies the budget on all 14. The largest violation of CHEHAB RL+RKS occurs on Hamming Distance (N=32), whose circuit consumes 271 bits, 17.8% above the budget. A violation is not a small loss of quality. Under fixed encryption parameters, a circuit that consumes more noise than the budget allows cannot be guaranteed to decrypt correctly; the only remedy is to choose larger encryption parameters, which increases both execution time and key size for the whole program. [TODO: Confirm with measured SEAL remaining-noise budgets or decryption checks that the violating circuits indeed fail, or that larger parameters are required.] The four benchmarks on which CHEHAB RL+RKS complies (Polynomial Regression, Linear Regression, Box Blur, and Sobel Gx) have shallow circuits that consume at most 219 bits under either system. For these benchmarks the budget is not binding, and CMO-RL behaves as an unconstrained co-optimizer of time and memory.

Execution time. fig. 1(b) shows that satisfying the budget does not cost CMO-RL execution time. CMO-

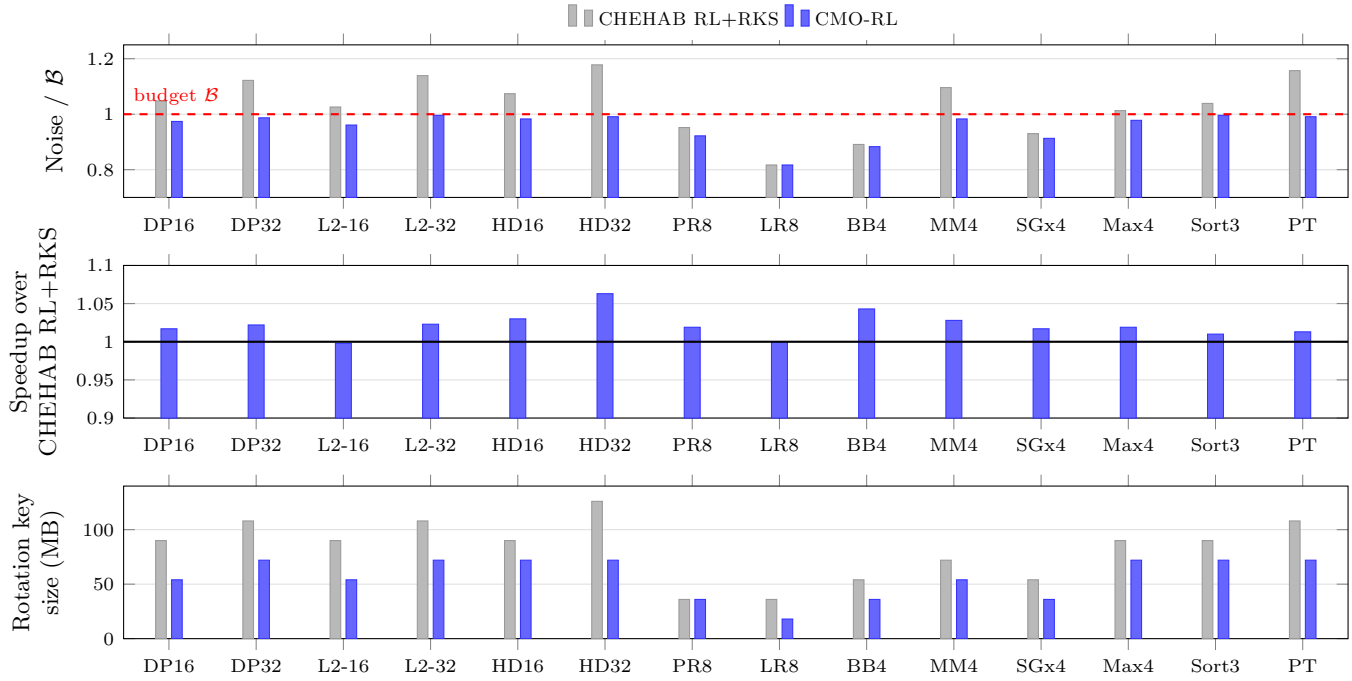


Figure 1: CMO-RL versus CHEHAB RL+RKS at noise budget $\mathcal{B} = 230$ bits and preference $\vec{w} = (0.9, 0.1)$. (a) Consumed noise normalized by $\mathcal{B}$; bars above the dashed line violate the budget. (b) Execution-time speedup of CMO-RL over CHEHAB RL+RKS (higher is better; values above 1 mean CMO-RL is faster). (c) Serialized rotation key size (lower is better). Illustrative placeholder data.

RL is faster than or as fast as CHEHAB RL+RKS on 13 of the 14 benchmarks, with a geometric-mean speedup of 1.02× (range 0.998×–1.063×; table 2). The one benchmark on which CMO-RL is slower, L2 Distance (N=16), differs by 1 ms (0.2%), which is [TODO: within the run-to-run variation of the measurements; report the standard deviation]. The largest speedup, 1.063×, appears on Hamming Distance (N=32), the benchmark where RKS decomposes the most rotations. We attribute this speedup to the order in which the two systems handle key memory. CHEHAB RL+RKS first produces a speed-optimized circuit and then replaces infrequent rotations with chains of power-of-two rotations; each chain executes several rotations where the original circuit executed one. CMO-RL reduces key memory while it chooses the vectorization, so it can select data layouts that reuse a small set of rotation offsets and need no rotation chains. [TODO: Support this explanation with operation counts: number of rotations inserted by RKS in CHEHAB RL+RKS vs. total rotations in CMO-RL.]

Rotation key memory. fig. 1(c) shows that CMO-RL requires fewer or equally many rotation keys on every benchmark. Over all 14 benchmarks, CMO-RL uses 44 keys (792 MB) against 64 keys (1152 MB) for CHEHAB RL+RKS, a reduction of 31% in total key size; the geometric-mean key-size ratio is 0.69 (table 2). The reduction is obtained even though CHEHAB RL+RKS applies a dedicated key-reduction pass and CMO-RL assigns only a weight of $w_{keys} = 0.1$ to memory. On Polynomial Regression (N=8), both systems require the same 2 keys, which suggests that 2 offsets are necessary for this circuit under both optimization strategies.

Why CHEHAB RL+RKS violates the budget. The noise model shows which operations consume the noise in the violating circuits. [TODO: Replace this paragraph with the measured breakdown; the explanation below is a hypothesis to be confirmed.] CHEHAB RL vectorizes aggressively because vector operations reduce its execution-cost model. Vectorization introduces rotations and plaintext masking multiplications, and in BFV each of these operations adds noise. RKS then lengthens the rotation sequences on the critical path by replacing single rotations with chains of rotations, which adds further key-switching noise. Neither step observes the noise budget. CMO-RL receives the noise budget and the current noise ratio as part of its state and is penalized when the circuit exceeds the budget, so it learns to stop applying noise-heavy rewrites once the remaining margin is small.

Summary. At a noise budget of 230 bits and a preference of $\vec{w} = (0.9, 0.1)$, CMO-RL satisfies the budget on all 14 benchmarks, whereas CHEHAB RL+RKS violates it on 10. CMO-RL achieves this with equal or lower execution time on 13 benchmarks (geometric-mean speedup 1.02×) and with 31% less rotation key memory

Table 1: Per-benchmark results at $\mathcal{B} = 230$ bits and $\vec{w} = (0.9, 0.1)$. Noise is the consumed noise in bits; ✗ marks a budget violation and ✓ marks compliance. Time is the execution time in ms. Keys is the number of distinct rotation keys; MB is their serialized size. Best value per row in bold. Illustrative placeholder data.

	CHEHAB RL+RKS					CMO-RL				
Benchmark	Noise		Time	Keys	MB	Noise		Time	Keys	MB
Dot Product (N=16)	241	✗	412	5	90	224	✓	**405**	**3**	**54**
Dot Product (N=32)	258	✗	655	6	108	227	✓	**641**	**4**	**72**
L2 Distance (N=16)	236	✗	**430**	5	90	221	✓	431	**3**	**54**
L2 Distance (N=32)	262	✗	790	6	108	229	✓	**772**	**4**	**72**
Hamming Dist. (N=16)	247	✗	980	5	90	226	✓	**951**	**4**	**72**
Hamming Dist. (N=32)	271	✗	1490	7	126	228	✓	**1402**	**4**	**72**
Poly. Regression (N=8)	219	✓	315	2	36	212	✓	**309**	2	36
Lin. Regression (N=8)	188	✓	140	2	36	188	✓	140	**1**	**18**
Box Blur (N=4)	205	✓	96	3	54	203	✓	**92**	**2**	**36**
Matrix Mul. (N=4)	252	✗	512	4	72	226	✓	**498**	**3**	**54**
Sobel Gx (N=4)	214	✓	530	3	54	210	✓	**521**	**2**	**36**
Maximum (N=4)	233	✗	1210	5	90	225	✓	**1188**	**4**	**72**
Sort (N=3)	239	✗	1380	5	90	229	✓	**1366**	**4**	**72**
Poly. Tree (100/50, d=5)	266	✗	2010	6	108	228	✓	**1985**	**4**	**72**

Table 2: Aggregate results over the 14 benchmarks of table 1. Geometric means are taken over per-benchmark ratios relative to CHEHAB RL+RKS. Illustrative placeholder data.

Metric	CHEHAB RL +RKS	CMO-RL
Budget violations	10/14	**0/14**
Max. overshoot (% of $\mathcal{B}$)	17.8%	**0%**
Geomean speedup	1.00×	**1.02**×
Lower-or-equal time	2/14	**13/14**
Total rotation keys	64	**44**
Total key size (MB)	1152	**792**
Geomean key-size ratio	1.00	**0.69**

in total. These results hold for one budget and one preference; section 1 [TODO: check reference] evaluates how CMO-RL behaves across budgets and preferences.